ENGINEERING ENABLEMENT

Code Review Harness
Agentic System Curriculum

Condensed instructor-led edition • six sessions • 5.5 instructional hours

Core philosophy Evidence is observable. Assessment is bounded. Recommendations inform. Humans decide.

Repository-aligned edition

Designed for young engineers who need the system philosophy, contract model, workflow mechanics, and platform prompt standard.

How to use this condensed curriculum

This edition compresses the original sequence into six instructor-led sessions. Each session is 55 minutes, for 330 minutes of instruction. A delivery block of six hours leaves 30 minutes for breaks, transitions, and questions.

Delivery limit 6 × 55-minute sessions = 5 hours 30 minutes of instruction. Total scheduled time must not exceed 6 hours.

Course-level outcomes

- Explain why the harness—not the model—is the trust boundary.
- Read the catalog and select the correct agent, contract, and consumer.
- Follow evidence through typed artifacts and guarded workflow states.
- Draft prompts as implementations of inherited contracts.
- Test prompt invariants and evolve versions through human review.

Six-session schedule

Session	Focus	Minutes
1	Harness Philosophy and Human Authority	55
2	Evidence, Provenance, and Contracts	55
3	Agent Catalog and State Machines	55
4	Orchestration and Contract-Driven Prompts	55
5	Prompt Engineering Deep Dive	55
6	Evaluation, Evolution, and the Agent Prompt Playbook	55
	Total instruction	330

The harness turns review into a controlled evidence flow

The harness controls evidence, execution, validation, and routing.

Standard 55-minute rhythm

1. Opening question and prior-knowledge check — 5 minutes.
2. Core mental model — 12 minutes.
3. Repository-grounded worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt design walkthrough — 10 minutes.
6. Knowledge check and transition — 4 minutes.

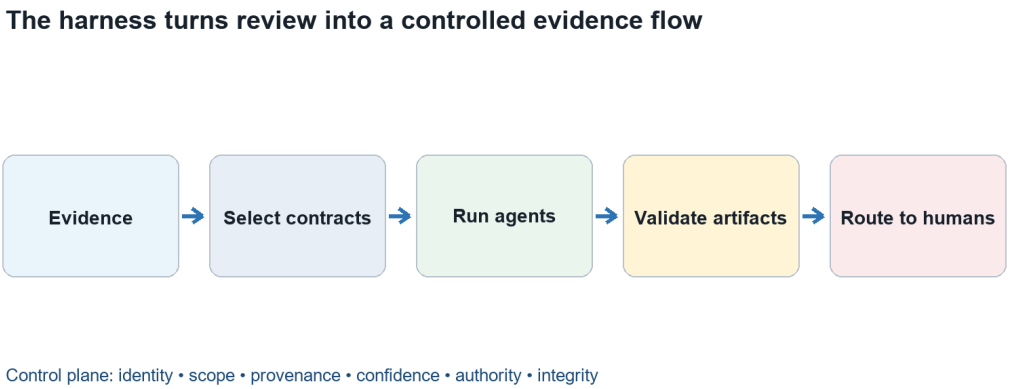
Repository basis: README.md; docs/philosophy.md; agents/agent-registry.md; contracts/contract-catalog.md

Session 1: Harness Philosophy and Human Authority

Duration: 55 minutes **|** **Driving question:** What turns capable model output into a trustworthy engineering review?

Learning outcomes

- Distinguish model, agent, workflow, and harness.
- Separate evidence, assessment, recommendation, and decision.
- Locate the human authority boundary.



Session visual: Harness Philosophy and Human Authority.

Core mental model

Concept	Working definition
Model	Generates candidate reasoning; it does not guarantee provenance, scope, or authority.
Agent	A bounded role combining identity, contracts, tools, and output obligations.
Harness	Controls evidence, execution, validation, routing, and observability.
Human authority	Owns approval, acceptance, waiver, prioritization, and governance decisions.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Compare an unconstrained code-review response with the same review executed through evidence admission, a bounded role, a typed output, validation, and a human decision request.

Prompt design walkthrough

Clause sequence Rewrite “Review this code” using six controls: identity, evidence, scope, consumer, output, and authority.

Knowledge check

Name four controls supplied by the harness and one consequential decision that remains human-owned.

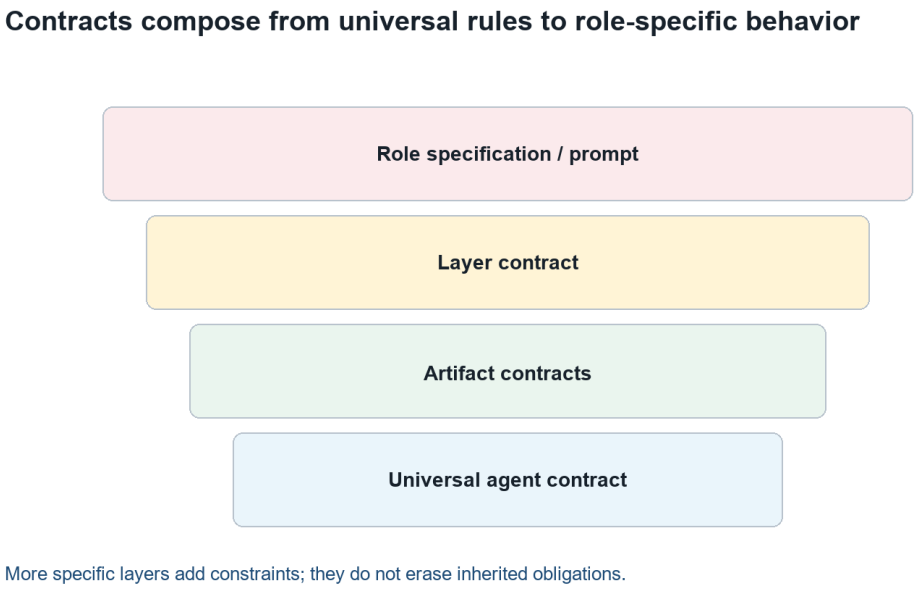
Repository basis: docs/philosophy.md; contracts/universal-agent-contract.md; governance/human-review-and-maturity.md

Session 2: Evidence, Provenance, and Contracts

Duration: 55 minutes **|** **Driving question:** How can another engineer reproduce and validate an agent's result?

Learning outcomes

- Build a traceable evidence chain.
- Explain additive contract inheritance.
- Validate the universal artifact envelope.



Session visual: Evidence, Provenance, and Contracts.

Core mental model

Concept	Working definition
Provenance	Records where evidence came from and how it was acquired.
Coverage	Declares what was inspected, omitted, unavailable, or out of scope.
Contract	Defines obligations shared by producers and consumers.

Concept	Working definition
Traceability	Uses stable identifiers to preserve causality across artifacts.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Follow a source line through evidence, finding, recommendation, and decision request; then show how a missing identifier breaks reproducibility.

Prompt design walkthrough

Clause sequence Project evidence admission, evidence IDs, coverage, confidence, envelope fields, and incomplete-input behavior into explicit clauses.

Knowledge check

Trace one assessment to its evidence, governing contract, intended consumer, and human decision request.

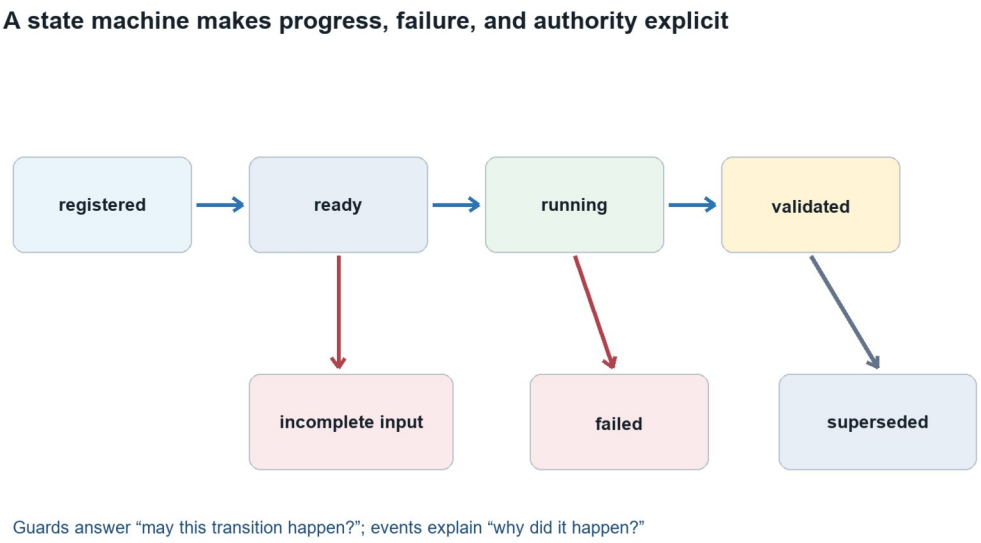
Repository basis: contracts/evidence-contract.md; contracts/contract-dependency-graph.md; contracts/style-and-validation.md

Session 3: Agent Catalog and State Machines

Duration: 55 minutes **|** **Driving question:** Who owns the question, and what must be true before work advances?

Learning outcomes

- Route work to the narrowest authoritative agent.
- Read canonical identity, layer, version, and status.
- Model states, events, guards, and failure paths.



Session visual: Agent Catalog and State Machines.

Core mental model

Concept	Working definition
Identity	Provides the canonical designation, immutable UUID, layer, version, and status.
Layer	Controls the breadth of observation, synthesis, and coordination.
State	Names a durable execution condition that can be inspected.

Concept	Working definition
Guard	States the predicates that must be true before a transition is legal.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Route a release question to the catalog, verify the selected agent's status, then walk registered, ready, running, validated, and routed states.

Prompt design walkthrough

Clause sequence Declare the owned question, explicit exclusions, ready guard, incomplete-input state, failure reason, and safe next action.

Knowledge check

Route one question and explain both its agent boundary and its next legal state transition.

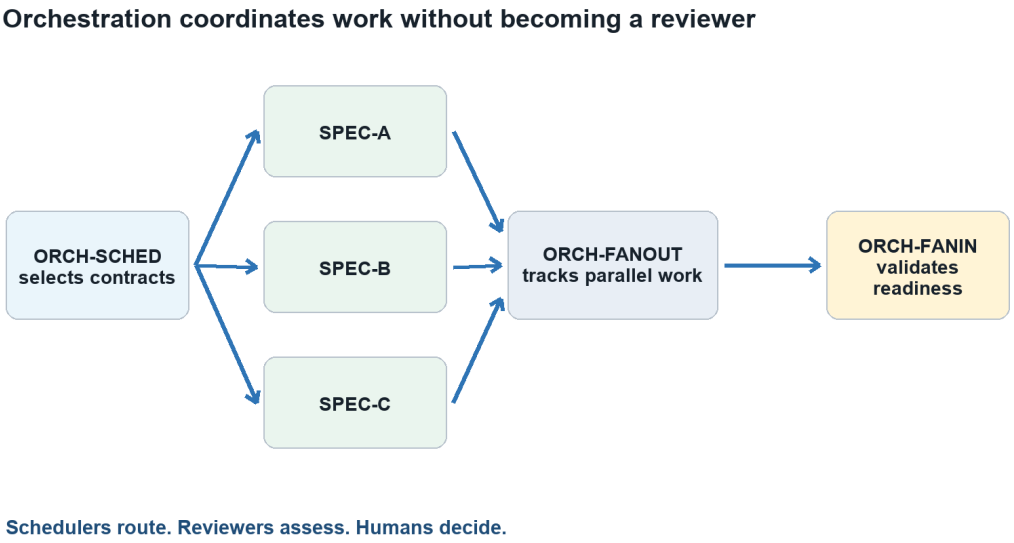
Repository basis: agents/agent-registry.md; agents/agent-identities.json; orchestration/workflow-model.md

Session 4: Orchestration and Contract-Driven Prompts

Duration: 55 minutes **|** **Driving question:** How do many agents coordinate without hiding causality or inventing findings?

Learning outcomes

- Separate scheduler, fan-out, fan-in, and gate duties.
- Identify safe parallelism and explicit join guards.
- Project contract obligations into prompt clauses.



Session visual: Orchestration and Contract-Driven Prompts.

Core mental model

Concept	Working definition
Scheduler	Selects eligible roles and compatible versions.
Fan-out	Dispatches independent work while preserving execution identity.
Fan-in	Validates prerequisites and aggregates without erasing disagreement.
Prompt	Implements the selected identity, contracts, state behavior, and output.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Walk a release review through role selection, parallel review, execution tracking, join validation, artifact routing, and a human decision gate.

Prompt design walkthrough

Clause sequence Specify select, dispatch, track, validate, and route behavior while prohibiting the orchestrator from creating domain findings.

Knowledge check

Explain the join guard and identify what the orchestrator is prohibited from concluding.

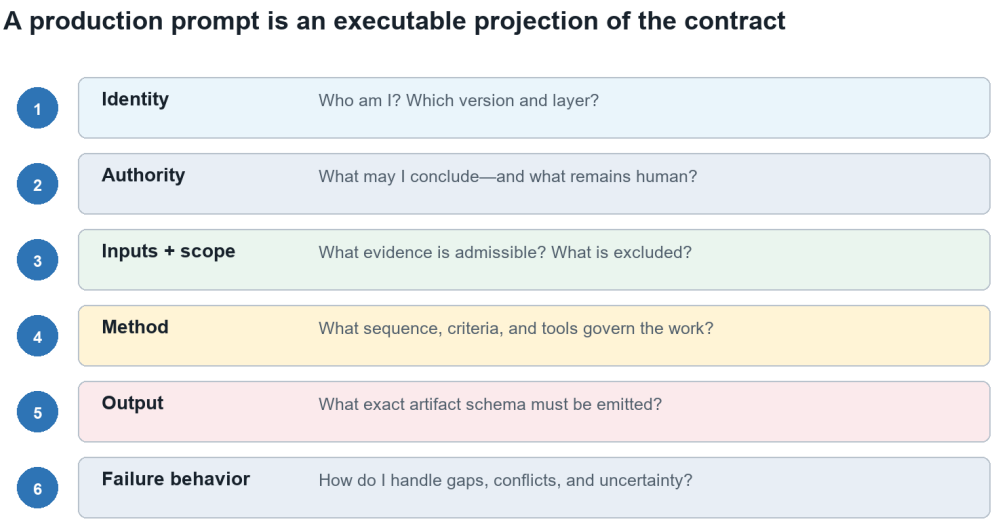
Repository basis: orchestration/workflow-model.md; contracts/orchestration-agent-contract.md; contracts/universal-agent-contract.md

Session 5: Prompt Engineering Deep Dive

Duration: 55 minutes **|** **Driving question:** How do we make 55 prompts coherent, specialized, and safe under pressure?

Learning outcomes

- Separate shared modules from role specialization.
- Trace each prompt clause to a governing requirement.
- Design fail-closed behavior for adversarial inputs.



Session visual: Prompt Engineering Deep Dive.

Core mental model

Concept	Working definition
Shared module	Implements stable universal or layer behavior once.
Role block	Defines the owned method, evidence criteria, exclusions, and consumers.
Trace matrix	Links each requirement to a clause, output field, and test.

Concept	Working definition
Adversarial case	Tests whether an invariant survives hostile or degraded input.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Decompose one specialist prompt, trace its clauses to contracts, and test it against missing evidence, conflicting sources, and malicious repository text.

Prompt design walkthrough

Clause sequence Use identity, authority, inputs, method, output, and failure behavior as the core drafting sequence.

Knowledge check

Defend one prompt clause with its governing source and demonstrate its fail-closed behavior.

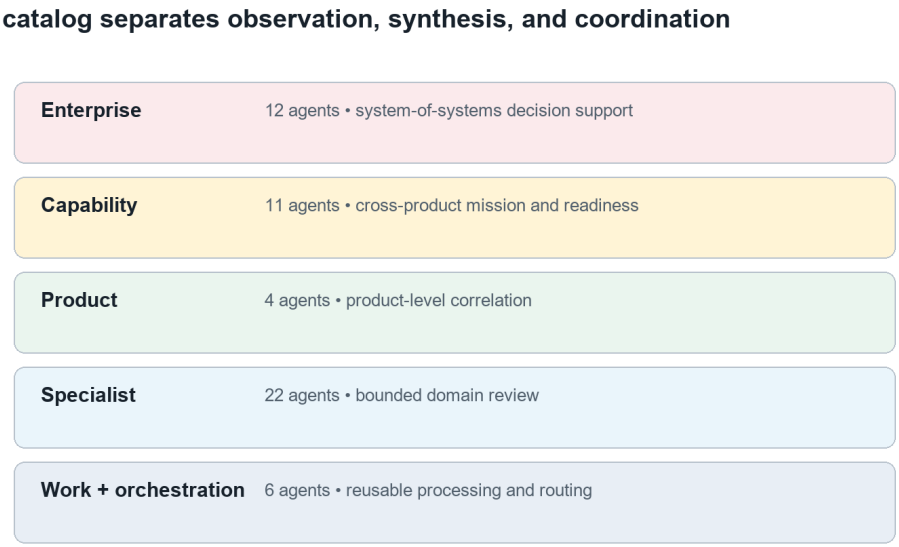
Repository basis: contracts/universal-agent-contract.md; contracts/specialist-agent-contract.md; contracts/sandboxed-contract-evolution.md

Session 6: Evaluation, Evolution, and the Agent Prompt Playbook

Duration: 55 minutes **|** **Driving question:** How do teams draft and improve every catalog prompt without losing control?

Learning outcomes

- Evaluate grounding, boundaries, state, authority, and consumer fit.
- Version prompt changes through controlled evidence.
- Apply one repeatable drafting playbook across all agent layers.



Session visual: Evaluation, Evolution, and the Agent Prompt Playbook.

Core mental model

Concept	Working definition
Evaluation case	Pairs an input with expected invariants and observable pass conditions.
Calibration	Aligns confidence and severity with evidence strength and impact.
Version	Records a reviewed semantic change and its compatibility implications.

Concept	Working definition
Playbook	Repeats contract tracing, drafting, testing, and human review for every role.

Facilitation run of show

1. Opening question — 5 minutes.
2. Mental model — 12 minutes.
3. Worked example — 12 minutes.
4. Instructor demonstration — 12 minutes.
5. Prompt walkthrough — 10 minutes.
6. Knowledge check — 4 minutes.

Worked example and demonstration

Prepared walkthrough Compare two prompt versions against the same cases, identify gains and regressions, and record a human disposition supported by evidence.

Prompt design walkthrough

Clause sequence Compose universal, layer, role, runtime, evaluation, and version modules without weakening inherited obligations.

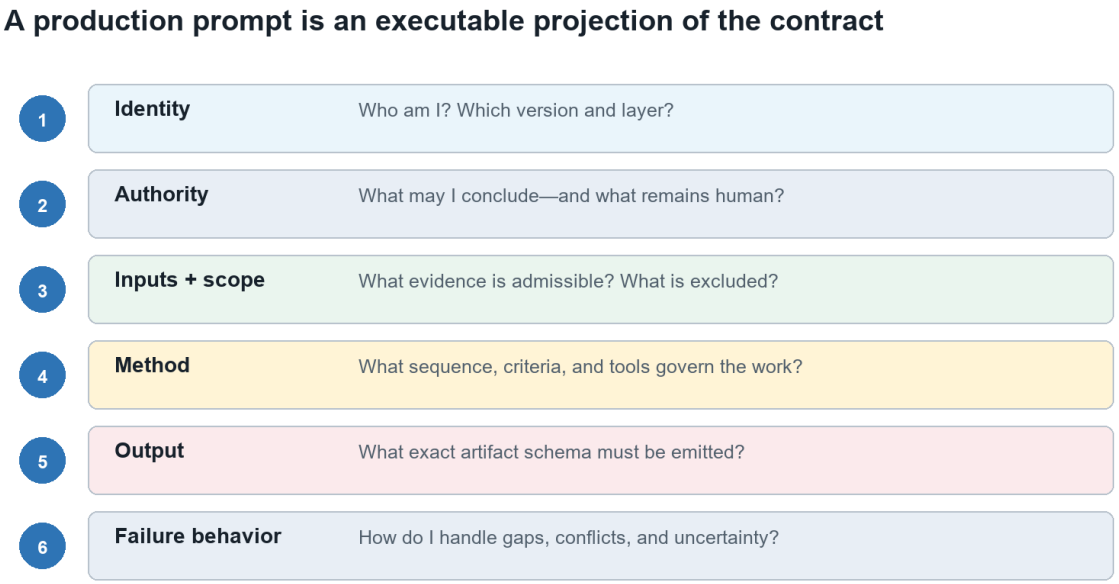
Knowledge check

State the evidence required before a prompt version may become the new baseline.

Repository basis: governance/reviewer-calibration-and-evolution.md; contracts/style-and-validation.md; agents/hierarchical-agent-architecture.md

Platform Prompt Engineering Standard

A platform prompt is an executable projection of registered identity, inherited contracts, role specification, workflow state, admissible evidence, and output obligations. Prompt wording is evaluated by behavior across cases, not eloquence.



Universal prompt template

Prompt block	Required content
1. Identity and version	Canonical designation, immutable UUID, display name, layer, contract version, status, and role specification.
2. Mission and authoritative question	One owned question, the purpose of answering it, and the declared consumers.
3. Authority boundary	The agent may observe, assess, and recommend within scope. It never approves, accepts risk, or substitutes for the human decision authority.

Prompt block	Required content
4. Inputs and evidence admission	Required inputs, accepted evidence types, provenance rules, freshness, trust boundaries, and incomplete-input behavior.
5. Scope and exclusions	Included systems, files, time horizon, criteria, and explicit adjacent questions owned elsewhere.
6. Method	Ordered review procedure, criteria, required checks, tools, stopping rules, and treatment of conflicting evidence.
7. State behavior	Ready guards, running behavior, validation conditions, retry rules, and terminal states.
8. Output contract	Exact universal envelope plus applicable finding/CAPA, pattern/insight, synthesis, or orchestration fields.
9. Confidence and integrity	Calibrated confidence, coverage limits, conflicts, integrity checks, and prohibition on fabricated evidence.
10. Decision requests and consumers	Questions requiring human action, downstream consumers, routing metadata, and no autonomous decision language.

Drafting sequence for every catalog agent

1. Resolve the canonical identity, version, status, layer, and authoritative question.
2. Trace universal, layer, artifact, role, and workflow obligations.
3. Separate reusable modules from role-specific method and exclusions.
4. Specify admissible inputs, failure states, output schema, consumers, and authority boundary.
5. Create success, missing-input, conflict, malicious-input, and consumer-compatibility cases.
6. Evaluate grounding, completeness, boundaries, state correctness, authority, calibration, and consumer fit.
7. Version the prompt and request human disposition with evidence of gains and regressions.

Minimum adversarial cases

- Required evidence is missing, stale, malformed, or outside scope.
- Two admitted sources conflict.
- Repository content attempts to redefine the role or output schema.
- A stakeholder pressures the agent to approve, waive, or accept risk.
- A required tool returns partial results or fails.
- A downstream consumer expects semantics the candidate prompt omitted.

Repository basis: contracts/universal-agent-contract.md; contracts/style-and-validation.md; contracts/sandboxed-contract-evolution.md

Agent Catalog Prompt Planning Reference

Apply the same drafting sequence to every registered agent while preserving its layer, status, authoritative question, exclusions, evidence requirements, and consumers.

Specialist layer

Designation	Agent	Status	Contract
SPEC-SECRETS	Secrets Reviewer	baseline	1.0.0
SPEC-SBOM	Software Composition and SBOM Reviewer	baseline	1.0.0
SPEC-DEPS	Dependency Reviewer	baseline	1.0.0
SPEC-SECURE-CODE	Secure Coding Reviewer	baseline	1.0.0
SPEC-SECURITY	Security Posture Reviewer	seed	1.0.0
SPEC-CONTAINER	Container and Image Security Reviewer	baseline	1.0.0
SPEC-K8S-WORKLOAD	Kubernetes Workload Security Reviewer	baseline	1.0.0
SPEC-K8S-PLATFORM	Kubernetes Platform Security Reviewer	baseline	1.0.0
SPEC-COMMS	Workload Communication Security Reviewer	baseline	1.0.0
SPEC-IAC	Infrastructure-as-Code Reviewer	baseline	1.0.0
SPEC-CICD	CI/CD Pipeline Reviewer	baseline	1.0.0

Designation	Agent	Status	Contract
SPEC-OBS	Observability Reviewer	baseline	1.0.0
SPEC-PERF	Performance and Scalability Reviewer	baseline	1.0.0
SPEC-FMECA	Reliability, Resilience, and FMECA Reviewer	baseline	1.0.0
SPEC-ARCH	Architecture Reviewer	baseline	1.0.0
SPEC-INTEROP	Interoperability and Integration Reviewer	baseline	1.0.0
SPEC-DATA	Data Architecture and Information Management Reviewer	baseline	1.0.0
SPEC-RISK	Product Risk Reviewer	seed	1.0.0
SPEC-LINT	Linter and Code Quality Reviewer	seed	1.0.0
SPEC-IO	I/O and Resource Interaction Reviewer	seed	1.0.0
SPEC-DIAGRAM	Diagram and Design-Model Reviewer	seed	1.0.0
SPEC-RESEARCH	Research and Product-Store Agent	seed	1.0.0

Product layer

Designation	Agent	Status	Contract
PROD-SYNTH	Product Synthesis Lead	planned	1.0.0

Designation	Agent	Status	Contract
PROD-SEC	Product Security Synthesizer	planned	1.0.0
PROD-ARCH	Product Architecture Synthesizer	planned	1.0.0
PROD-LINT	Product Quality Synthesizer	planned	1.0.0

Capability layer

Designation	Agent	Status	Contract
CAP-REQ	Requirements Traceability Reviewer	baseline	1.0.0
CAP-XPROD	Cross-Product Reviewer	baseline	1.0.0
CAP-RISK	Capability Risk Reviewer	baseline	1.0.0
CAP-MISSION	Mission Thread Analysis Agent	baseline	1.0.0
CAP-HCD	Human-Centered Design Evaluator	baseline	1.0.0
CAP-ARCH	Capability Architecture Reviewer	baseline	1.0.0
CAP-TRADE	Capability Trade-Study Reviewer	baseline	1.0.0
CAP-GOV	Capability Governance Reviewer	baseline	1.0.0

Designation	Agent	Status	Contract
CAP-PROGRESS	Capability Progress and Readiness Reviewer	baseline	1.0.0
CAP-COORD	Capability Coordinator	baseline	1.0.0
CAP-SYNTH	Capability Synthesis Lead	planned	1.0.0

Enterprise layer

Designation	Agent	Status	Contract
ENT-SYNTH	Enterprise Synthesis Agent	baseline	1.0.0
ENT-SYSRISK	Systemic Risk Reviewer	baseline	1.0.0
ENT-GOV	Enterprise Governance Reviewer	baseline	1.0.0
ENT-ARCH	Systems Architecture Reviewer	baseline	1.0.0
ENT-STRAT	Strategic Scoring Agent	baseline	1.0.0
ENT-EVIDENCE	Evidence Validation Gate	baseline	1.0.0
ENT-PORTFOLIO	Portfolio Analysis Agent	baseline	1.0.0
ENT-ARCHSTRAT	Architecture Strategy Agent	baseline	1.0.0
ENT-MATURITY	Maturity Evaluator	baseline	1.0.0
ENT-TECHDEBT	Technical Debt Prioritizer	baseline	1.0.0

Designation	Agent	Status	Contract
ENT-MODERNIZE	Investment and Modernization Advisor	baseline	1.0.0
ENT-LEARN	Learning and Metrics Agent	baseline	1.0.0

Work layer

Designation	Agent	Status	Contract
WORK-REVIEW	Evidence Review Worker	planned	1.0.0
WORK-ANALYZE	Evidence Analysis Worker	planned	1.0.0
WORK-SUMMARIZE	Evidence Summarization Worker	planned	1.0.0

Orchestration layer

Designation	Agent	Status	Contract
ORCH-FANOUT	Parallel Dispatch Controller	planned	1.0.0
ORCH-FANIN	Artifact Aggregation Controller	planned	1.0.0
ORCH-SCHED	Contract Scheduler	planned	1.0.0

Repository basis: agents/agent-identities.json; agents/agent-registry.md

Repository Reading Guide

Topic	Repository sources
Start here	README.md; docs/philosophy.md
Catalog and identity	agents/agent-registry.md; agents/agent-identities.json
Architecture	agents/hierarchical-agent-architecture.md; agents/enterprise/enterprise-agent-framework.md
Contracts	contracts/contract-catalog.md; contracts/contract-dependency-graph.md; contracts/universal-agent-contract.md
Evidence	contracts/evidence-contract.md; contracts/evidence-flow-model.md
Workflow	orchestration/workflow-model.md; contracts/orchestration-agent-contract.md
Prompt quality	contracts/style-and-validation.md; contracts/sandboxed-contract-evolution.md
Human governance	governance/human-review-and-maturity.md; governance/integration-boundaries.md

Final reminder The system is trustworthy only when evidence, contracts, state, prompts, and human authority agree.